

I. Objectifs

Dans cet atelier, les participants vont apprendre à **développer, configurer et tester un serveur MCP** en Python à partir d'un cas réel : un service d'information sur les conférences des recherches avancées recherches avancées et des ressources exploitables par un LLM (Claude, Copilot...).

L'objectif est de comprendre comment un serveur MCP expose :

- des **ressources** (resources)
- des **outils exécutables** (tools)
- et comment tout cela est rendu accessible à un client MCP via **STDIO**.

II. Travail demandé

1) Mise en place du projet Python :

Créer un nouveau projet ou vous pouvez travailler sur le projet de l'atelier précédent :

Étapes pour créer un nouveau projet :

1. Ouvrir le terminal : **uv venv**
2. Activez l'environnement : **.venv/Scripts/activate**
3. Initialiser un projet uv : **uv init**
4. Installez les dépendances : **uv add fastmcp django asgiref**

2) Implémentation du serveur "Conference Assistant"

Créer un script python **mcp_server.py** qui expose plusieurs tools permettant à un LLM d'accéder à des données métiers issus d'une base de données.

```
import os  
  
# Bibliothèque standard pour interagir avec le système d'exploitation, utilisée pour gérer les  
variables d'environnement.
```

```

import django
from fastmcp import FastMCP
# Importation de la classe FastMCP depuis le module fastmcp, utilisée pour créer un serveur
MCP rapide.

from asgiref.sync import sync_to_async
# Importation de sync_to_async pour convertir des fonctions synchrones en asynchrones,
compatible avec les ORM Django.

# Initialize Django environment
os.environ.setdefault("DJANGO_SETTINGS_MODULE",
"firstProject.settings")
django.setup()

# Importation des modèles Django après initialisation pour éviter des erreurs de configuration.
from ConferenceApp.models import Conference
from SessionApp.models import Session

# Create an MCP server
mcp = FastMCP("Conference Assistant")
# Lancement

if __name__ == "__main__":
    mcp.run(transport="stdio")

```

Votre serveur doit :

1. **Exposer au moins quatre outils (tools)** permettant d'interagir avec ces données :
 - a. un tool pour lister toutes les conférences disponibles.

```

# Décorateur pour définir un outil MCP exécutable de manière asynchrone, rendant cette fonction
accessible via le serveur MCP.
@mcp.tool()
async def list_conferences() -> str:
    """List all available conferences."""
    @sync_to_async #Ce décorateur permet d'appeler des méthodes synchrones de l'ORM
Django dans un contexte asynchrone, évitant les blocages.

    def _get_conferences():
# Fonction interne synchrone pour récupérer la liste des conférences depuis la base de données.
    return list(Conference.objects.all())

```

```

conferences = await _get_conferences()
# Appel asynchrone à la fonction interne pour obtenir les conférences, en attendant le résultat.
if not conferences:
    return "No conferences found."
return "\n".join([f"- {c.name} ({c.start_date} to {c.end_date})"
for c in conferences])
# Construit une chaîne formatée avec le nom et les dates de chaque conférence, séparées par
des sauts de ligne.

```

- b. un tool pour obtenir les détails d'une conférence spécifique par son nom.

```

@mcp.tool()
async def get_conference_details(name: str) -> str:
    """Get details of a specific conference by name."""
    @sync_to_async
    def _get_conference():
        try:
            return Conference.objects.get(name__icontains=name)
        except Conference.DoesNotExist:
            return None
        except Conference.MultipleObjectsReturned:
            return "MULTIPLE"

    conference = await _get_conference()
    if conference == "MULTIPLE":
        return f"Multiple conferences found matching '{name}'. Please be more specific."
    if not conference:
        return f"Conference '{name}' not found."

    return (
        f"Name: {conference.name}\n"
        f"Theme: {conference.get_theme_display()}\n"
        f"Location: {conference.location}\n"
        f"Dates: {conference.start_date} to {conference.end_date}\n"
        f"Description: {conference.description}"
    )

```

- c. un tool pour lister les sessions d'une conférence spécifique (ex : par nom de conférence).

```
@mcp.tool()
async def list_sessions(conference_name: str) -> str:
    """List sessions for a specific conference."""
    @sync_to_async
    def _get_sessions():
        try:
            conference =
Conference.objects.get(name__icontains=conference_name)
            return list(conference.sessions.all()), conference
        except Conference.DoesNotExist:
            return None, None
        except Conference.MultipleObjectsReturned:
            return "MULTIPLE", None

    result, conference = await _get_sessions()

    if result == "MULTIPLE":
        return f"Multiple conferences found matching
'{conference_name}'. Please be more specific."
    if conference is None:
        return f"Conference '{conference_name}' not found."
    sessions = result
    if not sessions:
        return f"No sessions found for conference
'{conference.name}'."
    session_list = []
    for s in sessions:
        session_list.append(
            f"- {s.title} ({s.start_time} - {s.end_time}) in
{s.room}\n"
            f"Topic: {s.topic}"
        )
    return "\n".join(session_list)
```

- d. un tool libre basé sur votre propre logique métier (ex : filtrer les conférences par thème ou date).

2. Tester avec l'inspecteur après, dans Claude Desktop ou Copilot.

```
.vscode > mcp.json > ...
1  {
2    "servers": {
3      "conferenceServer": {
4        "type": "stdio",
5        "command": "uv",
6        "args": [
7          "run",
8          "mcp_server.py"
9        ]
10     }
11   }
12 }
```

can you give more details about the IA conference?

✓ Optimizing tool selection...

> Ran `get_conference_details` – conferenceServer (MCP Server) ✓

IA Conference

- **Theme:** Computer Science & Artificial Intelligence
- **Location:** Lisbonne
- **Dates:** November 12-15, 2025
- **Description:** A conference about Artificial Intelligence in web developpement

Restore Checkpoint

what are the conferences in the database?

Here are the conferences in the database:

1. Conf 1 (October 3, 2025)
2. IA Conference (November 12-15, 2025)

🔄 ↩️ 👍 🗑️